# Graph Data Structure Study Guide

## 1. Detect a Cycle in an Undirected Graph

### Understanding Cycles in Undirected Graphs

A cycle in an undirected graph exists when there's a path that starts and ends at the same vertex, passing through at least 3 vertices. In undirected graphs, we need to be careful not to consider the immediate parent as creating a cycle.

### Graph Representation

```java
import java.util.*;

class Graph {
    private int vertices;
    private List<List<Integer>> adjacencyList;

    public Graph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        adjacencyList.get(u).add(v);
        adjacencyList.get(v).add(u); // Undirected graph
    }

    public List<Integer> getNeighbors(int vertex) {
        return adjacencyList.get(vertex);
    }

    public int getVertices() {
        return vertices;
    }
}
```

### 1.1 Cycle Detection Using DFS

DFS approach uses recursion to traverse the graph and keeps track of visited nodes and their parents.

**Algorithm:**

1. Start DFS from any unvisited vertex
2. Mark current vertex as visited
3. For each neighbor of current vertex:
   - If neighbor is not visited, recursively call DFS
   - If neighbor is visited and is not the parent of current vertex, cycle found

```java
public class CycleDetectionDFS {

    public boolean hasCycle(Graph graph) {
        boolean[] visited = new boolean[graph.getVertices()];

        // Check for cycle in each component
        for (int i = 0; i < graph.getVertices(); i++) {
            if (!visited[i]) {
                if (dfsHasCycle(graph, i, visited, -1)) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean dfsHasCycle(Graph graph, int vertex, boolean[] visited, int parent) {
        visited[vertex] = true;

        for (int neighbor : graph.getNeighbors(vertex)) {
            if (!visited[neighbor]) {
                if (dfsHasCycle(graph, neighbor, visited, vertex)) {
                    return true;
                }
            } else if (neighbor != parent) {
                // Found a back edge (cycle)
                return true;
            }
        }
        return false;
    }
}
```

**Time Complexity:** O(V + E) where V is vertices and E is edges **Space Complexity:** O(V) for visited array and recursion stack

## 1.2 Cycle Detection Using BFS

BFS approach uses a queue and tracks parent information to detect cycles.

**Algorithm:**

1. Use BFS traversal starting from any unvisited vertex
2. For each vertex, store its parent information
3. If we encounter a visited vertex that is not the parent, cycle exists

```java
public class CycleDetectionBFS {

    public boolean hasCycle(Graph graph) {
        boolean[] visited = new boolean[graph.getVertices()];

        for (int i = 0; i < graph.getVertices(); i++) {
            if (!visited[i]) {
                if (bfsHasCycle(graph, i, visited)) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean bfsHasCycle(Graph graph, int start, boolean[] visited) {
        Queue<Integer> queue = new LinkedList<>();
        int[] parent = new int[graph.getVertices()];

        queue.offer(start);
        visited[start] = true;
        parent[start] = -1;

        while (!queue.isEmpty()) {
            int current = queue.poll();

            for (int neighbor : graph.getNeighbors(current)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    parent[neighbor] = current;
                    queue.offer(neighbor);
                } else if (parent[current] != neighbor) {
                    // Found a cycle
                    return true;
                }
            }
        }
        return false;
    }
}
```

**Time Complexity:** O(V + E) **Space Complexity:** O(V) for visited array, parent array, and queue

## 2. Bipartite Graph

## Understanding Bipartite Graphs

A bipartite graph is a graph whose vertices can be divided into two disjoint sets such that no two vertices within the same set are adjacent. In other words, we can color the graph using only two colors such that no adjacent vertices have the same color.

**Properties:**

- Contains no odd-length cycles
- Can be 2-colored
- Often represents relationships between two different types of entities

## 2.1 Bipartite Check Using BFS

BFS approach uses a queue and tries to 2-color the graph level by level.

**Algorithm:**

1. Start BFS from any unvisited vertex and color it with color 0
2. Color all neighbors with opposite color (color 1)
3. If we try to color a vertex that's already colored with a different color, graph is not bipartite

```java
public class BipartiteBFS {

    public boolean isBipartite(Graph graph) {
        int[] color = new int[graph.getVertices()];
        Arrays.fill(color, -1); // -1 means uncolored

        // Check each component
        for (int i = 0; i < graph.getVertices(); i++) {
            if (color[i] == -1) {
                if (!bfsCheckBipartite(graph, i, color)) {
                    return false;
                }
            }
        }
        return true;
    }

    private boolean bfsCheckBipartite(Graph graph, int start, int[] color) {
        Queue<Integer> queue = new LinkedList<>();
        queue.offer(start);
        color[start] = 0; // Start with color 0

        while (!queue.isEmpty()) {
            int current = queue.poll();

            for (int neighbor : graph.getNeighbors(current)) {
                if (color[neighbor] == -1) {
                    // Color with opposite color
                    color[neighbor] = 1 - color[current];
                    queue.offer(neighbor);
                } else if (color[neighbor] == color[current]) {
                    // Same color for adjacent vertices
                    return false;
                }
            }
        }
        return true;
    }
}
```

**Time Complexity:** O(V + E) **Space Complexity:** O(V) for color array and queue

## 2.2 Bipartite Check Using DFS

DFS approach uses recursion to traverse and color the graph.

**Algorithm:**

1. Start DFS from any unvisited vertex and color it
2. Recursively color all neighbors with opposite color
3. If we encounter a conflict in coloring, graph is not bipartite

```java
public class BipartiteDFS {

    public boolean isBipartite(Graph graph) {
        int[] color = new int[graph.getVertices()];
        Arrays.fill(color, -1);

        for (int i = 0; i < graph.getVertices(); i++) {
            if (color[i] == -1) {
                if (!dfsCheckBipartite(graph, i, color, 0)) {
                    return false;
                }
            }
        }
        return true;
    }

    private boolean dfsCheckBipartite(Graph graph, int vertex, int[] color, int currentColor) {
        color[vertex] = currentColor;

        for (int neighbor : graph.getNeighbors(vertex)) {
            if (color[neighbor] == -1) {
                // Color with opposite color
                if (!dfsCheckBipartite(graph, neighbor, color, 1 - currentColor)) {
                    return false;
                }
            } else if (color[neighbor] == color[vertex]) {
                // Same color for adjacent vertices
                return false;
            }
        }
        return true;
    }
}
```

**Time Complexity:** O(V + E) **Space Complexity:** O(V) for color array and recursion stack

## Example Usage

```java
public class GraphExample {
    public static void main(String[] args) {
        // Create a graph with 4 vertices
        Graph graph = new Graph(4);

        // Add edges to create a cycle: 0-1-2-3-0
        graph.addEdge(0, 1);
        graph.addEdge(1, 2);
        graph.addEdge(2, 3);
        graph.addEdge(3, 0);

        // Test cycle detection
        CycleDetectionDFS cycleDFS = new CycleDetectionDFS();
        CycleDetectionBFS cycleBFS = new CycleDetectionBFS();

        System.out.println("Has cycle (DFS): " + cycleDFS.hasCycle(graph));
        System.out.println("Has cycle (BFS): " + cycleBFS.hasCycle(graph));

        // Test bipartite
        BipartiteDFS bipartiteDFS = new BipartiteDFS();
        BipartiteBFS bipartiteBFS = new BipartiteBFS();

        System.out.println("Is bipartite (DFS): " + bipartiteDFS.isBipartite(graph));
        System.out.println("Is bipartite (BFS): " + bipartiteBFS.isBipartite(graph));
    }
}
```

---

## Quiz Questions

**Question 1:** In cycle detection for undirected graphs using DFS, why do we need to track the parent vertex?

A) To optimize the algorithm's performance
B) To avoid considering the edge we just came from as a cycle
C) To maintain the path traversed so far
D) To implement backtracking

**Correct Answer: B**

**Question 2:** What is the time complexity of detecting a cycle in an undirected graph using BFS?

A) $O(V^2)$
B) $O(V + E)$

C) $O(V \times E)$
D) $O(E^2)$

**Correct Answer: B**

**Question 3:** A bipartite graph can be characterized by which of the following properties?

A) It contains at least one odd-length cycle
B) It can be colored using exactly 3 colors
C) It contains no odd-length cycles
D) All vertices have the same degree

**Correct Answer: C**

**Question 4:** In the bipartite check algorithm using DFS, what does it mean when we encounter a neighbor that has the same color as the current vertex?

A) The graph is bipartite
B) The graph is not bipartite
C) We need to try a different coloring
D) We found a valid 2-coloring

**Correct Answer: B**

**Question 5:** Which data structure is primarily used in the BFS approach for both cycle detection and bipartite checking?

A) Stack
B) Priority Queue
C) Queue
D) Hash Set

**Correct Answer: C**

---

## Practice Coding Questions

**Question 1:** Given an undirected graph represented as an adjacency list, implement a function that not only detects if there's a cycle but also returns the vertices that form the cycle. If no cycle exists, return an empty list.

**Question 2:** Implement a function to determine if a given undirected graph can be divided into two sets such that the sum of vertices in both sets is equal, and the graph remains bipartite after removing one edge.

**Question 3:** Given a bipartite graph, implement a function to find the maximum number of edges that can be added to the graph while keeping it bipartite.

**Question 4:** Write a function that takes an undirected graph and returns the minimum number of edges that need to be removed to make the graph acyclic (i.e., convert it to a forest).

**Question 5:** Implement a function to check if an undirected graph is a complete bipartite graph (a bipartite graph where every vertex in one set is connected to every vertex in the other set).